Practical No. 4 – Binary Search Tree (BST) using Python

Name :- Shrutika Sachin Manedeshmukh **Roll NO :-** 240 **Class :-** SE AIML **Batch :-** S2

Problem Statement: Binary Search Tree -- CO1, CO2, CO3

Implement binary search tree and perform following operations

a. Insert (Handle insertion of duplicate entry)

b. Delete

c. Search

d. Display tree (Traversal)

e. Display - Depth of tree

f. Display - Mirror image g. Create a copy

h. Display all parent nodes with their child nodes

i. Display leaf nodes

j. Display tree level wise

Code

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


# Insert
def insert(root, data):
    if root is None:
        return Node(data)


    if data < root.data:
        root.left = insert(root.left, data)
```

```
    else:
        root.right = insert(root.right, data)

    return root
# Search
def search(root, data):
    if root is None:
        return False
    if root.data == data:
        return True

    if data < root.data:
        return search(root.left, data)
    else:
        return search(root.right, data)
# Delete
def delete(root, data):
    if root is None:
        return root

    if data < root.data:
        root.left = delete(root.left, data)

    elif data > root.data:
        root.right = delete(root.right, data)
    else:
        if root.left is None:
            return root.right
```

```
        if root.right is None:
            return root.left

        temp = root.right
        while temp.left:
            temp = temp.left
        root.data = temp.data
        root.right = delete(root.right, temp.data)
    return root
# Inorder
def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)

# Preorder
def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)

# Postorder
def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")
```

```
# Depth
def depth(root):
    if root is None:
        return 0

    return max(depth(root.left), depth(root.right)) + 1

# Mirror
def mirror(root):
    if root:
        root.left, root.right = root.right, root.left
        mirror(root.left)
        mirror(root.right)

# Copy
def copy_tree(root):
    if root is None:
        return None

    new_root = Node(root.data)
    new_root.left = copy_tree(root.left)
    new_root.right = copy_tree(root.right)

    return new_root

# Parent nodes with children
def parent_nodes(root):
    if root:
```

```
        if root.left or root.right:
            print("Parent:", root.data, end="  ")

            if root.left:
                print("Left Child:", root.left.data, end="  ")

            if root.right:
                print("Right Child:", root.right.data, end="  ")

            print()

        parent_nodes(root.left)
        parent_nodes(root.right)

# Leaf nodes
def leaf_nodes(root):
    if root:
        if root.left is None and root.right is None:
            print(root.data, end=" ")

        leaf_nodes(root.left)
        leaf_nodes(root.right)

# Level-wise display
def level_order(root):
    if root is None:
        return

    queue = [root]
```

```
    while queue:
        temp = queue.pop(0)
        print(temp.data, end=" ")

        if temp.left:
            queue.append(temp.left)

        if temp.right:
            queue.append(temp.right)

# ---------------- MAIN ----------------

root = None

# Insert elements
values = [50, 30, 70, 20, 40, 60, 80]

for value in values:
    root = insert(root, value)

print("Inorder Traversal:")
inorder(root)

print("\nPreorder Traversal:")
preorder(root)

print("\nPostorder Traversal:")
postorder(root)
```

```
print("\n\nSearch 40:")
if search(root, 40):
    print("40 Found")
else:
    print("40 Not Found")

print("\nDepth of Tree:")
print(depth(root))

print("\nParent Nodes with Children:")
parent_nodes(root)

print("Leaf Nodes:")
leaf_nodes(root)

print("\nLevel Wise:")
level_order(root)

# Delete
root = delete(root, 30)

print("\n\nAfter deleting 30:")
inorder(root)

# Create Copy
copy_root = copy_tree(root)

print("\n\nCopied Tree:")
```

```
inorder(copy_root)

# Mirror

mirror(root)


print("\n\nMirror Tree:")

inorder(root)
```

Output

```
Inorder Traversal:
20 30 40 50 60 70 80

Preorder Traversal:
50 30 20 40 70 60 80

Postorder Traversal:
20 40 30 60 80 70 50

Search 40:
40 Found

Depth of Tree:3

Parent Nodes with Children:

Parent: 50  Left Child: 30  Right Child: 70

Parent: 30  Left Child: 20  Right Child: 40

Parent: 70  Left Child: 60  Right Child: 80

Leaf Nodes:  20 40 60 80

Level Wise:    50 30 70 20 40 60 80

After deleting 30:   20 40 50 60 70 80

Copied Tree:20 40 50 60 70 80

Mirror Tree:80 70 60 50 40 20
```